Phase 1: TSP Instance Loading & Distance Matrix

An Inquiry-Based Systems Programming Assignment

Architecture First. Code Second.

0 Contents

1 Critique of the Two Proposal Documents **2**
- 1.1 Proposal A (Doc 1: "Inquiry-Based Coding Assignment") . . . 2
- 1.2 Proposal B (Doc 2: "Memory & Parsing Audit") . . . 3

2 The Revised Assignment: Phase 1 Foundations **4**
- 2.1 What you are building . . . 4
- 2.2 File layout you will create . . . 4

3 Design Decision 1: Matrix Storage Layout **5**
- 3.1 The three options . . . 5
- 3.2 Comparison table . . . 5
- 3.3 The hardware reason flat arrays win . . . 6
- 3.4 The access macro and why it works . . . 6

4 Design Decision 2: Distance Matrix Precision **8**
- 4.1 The type candidates . . . 8
- 4.2 The GPU constant-memory calculation (done correctly) . . . 8
- 4.3 The `uint16_t` option evaluated . . . 8

5 Design Decision 3: Full Matrix vs. Packed Symmetric Storage **10**
- 5.1 Two representations . . . 10

6 Design Decision 4: Parsing Strategy **11**
- 6.1 The two parsing approaches . . . 11
- 6.2 Comparison . . . 11
- 6.3 One-pass vs. two-pass allocation . . . 11

7 The Complete API Specification **13**
- 7.1 The data structure (`include/instance.h`) . . . 13

8 The TSPLIB File Format (Narrow Subset) **18**
- 8.1 Format you will support . . . 18
- 8.2 Test fixture: `square_4.tsp` . . . 18

9 C Implementation Guide **20**
9.1 Step 1: Struct, macro, and static initialiser . . . 20
9.2 Step 2: Implement `tsp_instance_free` . . . 20
9.3 Step 3: Skeleton of `tsp_instance_load` . . . 20
9.4 Step 4: Implement `tsp_build_distance_matrix` . . . 21
9.5 Step 5: The Makefile . . . 22

10 Testing: Writing Plain-C Unit Tests **23**
10.1 Malformed fixture files . . . 24
10.2 Running under Valgrind . . . 25

11 Test Matrix: What You Are Verifying **26**

12 Questions You Must Answer in Writing **27**

13 Implementation Order Summary **28**

14 Reflection: After Everything Passes **29**

15 Optional Extensions **29**

1 Critique of the Two Proposal Documents

Before introducing the revised assignment, it is worth auditing the two source documents against your stated learning goals. Your goals, ranked, are:

1. **Primary:** System design — understand architecture, analyse alternatives, weigh tradeoffs.
2. **Secondary:** Learn to implement in C at a level that builds real familiarity.
3. **Tertiary:** Arrive at a working, memory-safe, testable Phase 1 baseline.

1.1 Proposal A (Doc 1: "Inquiry-Based Coding Assignment")

What Proposal A does well

- Strong on the *why* behind every design decision. It explains *why* flat arrays beat `double**`, not just which to use.
- Correctly identifies that constant-memory is *not* a reliable target and gives exact arithmetic to prove it ($128 \times 128 \times 8 = 128\,\text{KB}$ vs. $64\,\text{KB}$ budget).
- Provides a clean, explicit API with named functions, separating concerns properly.
- Offers extension paths for ambitious work without muddying the main path.
- Includes a concrete testing matrix (P-01 through M-01), which is essential for a verification mindset.

What Proposal A fails to deliver for your goals

- **Functions are named but not specified.** You see `tsp_instance_load(const char *path, TSPInstance *out)` but nowhere is there a description of: what arguments mean, what return values mean, what the function guarantees on success, what it guarantees on failure, or what state `out` is left in when something goes wrong. For someone learning C, this is the exact information that needs to be explicit.
- **No hardware grounding.** It talks about flat arrays being good for GPU mirroring but does not explain *why* at the memory-bus level. It mentions cache locality without explaining spatial vs. temporal locality, prefetching, or cache lines.
- **Inquiry boxes are vague.** "Compare alternatives" appears often, but the actual comparison is not scaffolded with enough concrete data to force a conclusion. A learner can read the inquiries and still not know how to decide.
- **No file-structure scaffolding.** A beginning systems programmer looking at this needs to know: where does each function live, what headers does each file include, and in what order should things be written. That guidance is absent.

1.2 Proposal B (Doc 2: "Memory & Parsing Audit")

What Proposal B does well

- Hardware framing is strong: it mentions syscall count, cache prefetchers, and `cudaMemcpy` count explicitly.
- The `sqrt` vs. `sqrtf` inquiry is excellent and points at a real C pitfall that most beginners miss.
- Modular naming (Module 1–4) makes the assignment feel manageable.
- The symmetry question at the end is a genuine design fork, not rhetorical.

What Proposal B fails to deliver for your goals

- **The constant-memory analysis is exactly backwards.** Proposal B presents `float` as the solution to the 64 KB constraint for 128 cities. Let's check: $128 \times 128 \times 4 = 65{,}536$ bytes = 64 KB exactly. That is the *nominal hardware limit*, not comfortable margin. Proposal A correctly identifies this. Designing a system where you pray that the hardware limit is not 63.5 KB in practice is not architecture—it is wishful thinking.
- **The `uint16_t` option is raised and never resolved.** Integer weights for Euclidean coordinates with decimal values require explicit rounding decisions (floor? round? TSPLIB rounding?) that have non-trivial effects on tour quality. Raising this option without providing the intellectual tools to evaluate it is harmful.
- **Function signatures are not given at all.** The assignment asks you to implement `tsp_instance_load`, `tsp_instance_free`, and `tsp_build_distance_matrix` but does not specify return types, argument meanings, error semantics, or invariants.
- **The modular structure is not connected.** Modules 1–4 read like four separate assignments. There is no scaffolding that shows how the pieces connect: what struct one module produces, which pointer the next module consumes.
- **No test structure is given.** "Write a test runner" is not enough guidance for a C learner to understand how to write a minimal unit test in plain C without a framework.

Synthesis verdict

Neither proposal alone is sufficient for your goals. Proposal A has better design reasoning; Proposal B has better hardware grounding. The revised assignment below takes the strongest elements of both, corrects the constant-memory mistake, and adds the one thing both are missing: **complete function contracts that specify inputs, outputs, and behavioral guarantees in enough detail to write the code from.**

2 The Revised Assignment: Phase 1 Foundations

2.1 What you are building

You are implementing a data-loading and matrix-construction layer for a Traveling Salesman Problem (TSP) solver in C99. This layer sits at the foundation of a system that will later run a Genetic Algorithm on the CPU, then migrate the hottest kernel to CUDA.

The system pipeline for Phase 1 is:

`[.tsp file on disk]` ⟶ **`parser`** ⟶ **`coordinate arrays`** ⟶ **`distance matrix`** ⟶ `[solver, later]`

Phase 1 delivers exactly three things:

1. A struct that holds a TSP instance in memory.
2. A function that parses a TSPLIB-format file into that struct.
3. A function that fills the distance matrix from the parsed coordinates.

2.2 File layout you will create

```
tsp/
  include/
    instance.h <- struct definition, macro, function declarations
  src/
    instance.c <- parsing, matrix construction, cleanup
  tests/
    test_instance.c <- unit tests (no framework needed)
    fixtures/
      square_4.tsp <- 4-city known geometry test
      malformed_bad_coord.tsp <- line with letters instead of numbers
      malformed_missing_section.tsp <- no NODE_COORD_SECTION
  Makefile
```

You will implement in this order: struct → cleanup → parser → matrix → tests.

3 Design Decision 1: Matrix Storage Layout

This is the most fundamental architectural decision in Phase 1.

3.1 The three options

You are choosing how to represent a 2D grid of $n \times n$ distances in C memory. Here are the three serious candidates.

Option A: Array of pointers (`double **dist`)

```c
double **dist = malloc(n * sizeof(double *)); // 1 malloc
for (int i = 0; i < n; i++)
    dist[i] = malloc(n * sizeof(double)); // n more mallocs
// Access: dist[i][j]
```

Option B: Flat 1D array (`double *dist`)

```c
double *dist = malloc(n * n * sizeof(double)); // 1 malloc
// Access: dist[i * n + j]
// Or with a macro: DIST(inst, i, j)
```

Option C: No stored matrix — compute on demand

```c
// No dist field at all.
// Call: sqrt((x[i]-x[j])^2 + (y[i]-y[j])^2) each time.
```

3.2 Comparison table

Work through this table before reading the recommendation

Fill in each cell before reading further. Forced thinking before revealed answer.

Property	A: double**	B: flat double*	C: on demand
Number of `malloc` calls	$n+1$	1	0
Memory is contiguous?	No	Yes	N/A
Rows adjacent in memory?	No	Yes	N/A
`free()` complexity	loop + outer free	single free	nothing
Copy to GPU (`cudaMemcpy`)	n calls	1 call	N/A
Access syntax	`dist[i][j]`	`dist[i*n+j]`	function call
Spatial cache locality	Poor	Excellent	N/A
Work per distance query	O(1)	O(1)	1 sqrt + arithmetic
Repeated access cost	Equal	Equal	Each call is paid again

3.3 The hardware reason flat arrays win

When the CPU reads memory, it does not load one byte—it loads a **cache line** (typically 64 bytes on modern hardware). If your data is contiguous, loading one element brings its neighbours into cache automatically. This is called *spatial locality*, and it means the hardware prefetcher can predict your next access and load it before you ask.

With `double**`, each row is a separate allocation somewhere in the heap. Row i and row $i+1$ are not adjacent. When you finish row i and start row $i+1$, the prefetcher cannot help you. This causes a cache miss for every row boundary.

For GPU transfer: `cudaMemcpy` expects a single contiguous source address and a byte count. A flat array gives you that in one call. A `double**` requires either n individual copies (expensive overhead) or a reformatting step before transfer.

Decision: Use Option B

Flat 1D array, accessed with a macro. Reason: single allocation, contiguous memory, one-call GPU copy, simple ownership model. Option A is the beginner trap in C. Option C is only interesting if memory is extremely constrained and access is sparse—not true for a dense GA solver.

3.4 The access macro and why it works

In C, a 2D index (i, j) in a row-major flat array of width n is computed as $i \cdot n + j$. You will expose this through a macro so that code that uses the matrix does not need to know the internal layout:

```
#define DIST(inst, i, j) ((inst)->dist[(i) * (inst)->n + (j)])
```

The extra parentheses around every argument are mandatory. Without them, a call like `DIST(inst, a+1, b)` would expand to `inst->dist[a+1 * n + b]`, which is wrong due to operator precedence. Parenthesise macro arguments defensively, always.

4 Design Decision 2: Distance Matrix Precision

4.1 The type candidates

Type	Bytes	Range	Decimal digits	128^2 footprint	Notes
`double`	8	$\pm 1.8 \times 10^{308}$	$\approx$ 15–16	128 KB	IEEE 754 double precision
`float`	4	$\pm 3.4 \times 10^{38}$	$\approx$ 6–7	64 KB	IEEE 754 single precision
`uint16_t`	2	0–65535	integer only	32 KB	Requires explicit rounding

4.2 The GPU constant-memory calculation (done correctly)

Both source proposals discuss GPU `__constant__` memory. You should understand this, but do not let it drive your Phase 1 decision.

Nominal constant memory budget on NVIDIA hardware: 64 KB.

$$
\begin{aligned}
128 \times 128 \times 8 &= 131{,}072 \text{ bytes} = 128 \text{ KB} \quad \text{(double — too large)} \\
128 \times 128 \times 4 &= 65{,}536 \text{ bytes} = 64 \text{ KB} \quad \text{(float — exactly the limit)} \\
128 \times 128 \times 2 &= 32{,}768 \text{ bytes} = 32 \text{ KB} \quad \text{(uint16_t — fits comfortably)}
\end{aligned}
$$

The constant-memory trap

Proposal B suggests using `float` because $128^2 \times 4 = 64$ KB fits the constant memory budget. This reasoning has two problems:

1. 64 KB is the *nominal* maximum. In practice, the CUDA runtime and driver also use constant memory for internal variables. Your available budget is typically less than 64 KB.
2. Constant memory is not a scratchpad you can always use. The GPU broadcasts from constant memory efficiently *only when all threads in a warp access the same address simultaneously*. A GA kernel where each thread follows a different tour accesses distances non-uniformly, which means constant memory may actually be *slower* than L2 cache for this application.

The right design choice is: **use what is correct and clean in Phase 1**. We will decide GPU memory placement in Phase 3, when we have profiling data.

4.3 The `uint16_t` option evaluated

Proposal B raises `uint16_t`. This is a valid option for solvers that use TSPLIB instances with *integer weights*, but not for Euclidean instances. The reason:

Euclidean distance between $(0, 0)$ and $(1, 1)$ is $\sqrt{2} \approx 1.41421356\ldots$. To store this as `uint16_t = 1` you must round. TSPLIB defines a specific rounding convention (`NINT`, which is nearest-integer). Using it changes the problem slightly. For Phase 1 you are building a general Euclidean solver, so discard `uint16_t` unless you later decide to implement TSPLIB integer-weight mode.

Decision: Use `double` for Phase 1

- `double` is the natural type for Euclidean geometry in C. `sqrt` returns `double`; coordinates parsed with `%lf` are `double`.
- No implicit casts, no rounding decisions, no precision surprises.
- The CPU baseline is for correctness first. Memory pressure optimisation is a Phase 3 concern.
- If you intentionally choose `float` for Phase 1, you must document that you understand the above and explain why the reduced precision is acceptable.

5 Design Decision 3: Full Matrix vs. Packed Symmetric Storage

The Euclidean distance matrix is symmetric: $D_{ij} = D_{ji}$. You can exploit this.

5.1 Two representations

Full $n \times n$ matrix:

- Memory: n^2 entries.
- Index: `dist[i*n + j]` for any (i, j).
- Diagonal: $D_{ii} = 0$ explicitly stored.

Packed upper-triangular (only $j > i$):

- Memory: $n(n-1)/2$ entries.
- Index: requires a formula. For $i < j$: index $= i \cdot n - i(i+1)/2 + j - i - 1$.
- Diagonal: not stored; always implicitly 0.

Property	Full $n \times n$	Packed upper triangle
Memory for $n = 128$, `double`	128 KB	63.5 KB
Memory for $n = 512$, `double`	2 MB	$\approx$ 1 MB
Index expression	`i*n+j`	$in - i(i+1)/2 + j - i - 1$
Can use same DIST macro?	Yes	No; needs new macro
Access pattern for GA	Uniform	Uniform
Copy to GPU	Trivial	Trivial
Debuggability	Easy (print as grid)	Hard (packed blob)

Before reading the decision, answer this

If you pack only the upper triangle and the solver code calls `DIST(inst, 5, 2)` where $5 > 2$, what happens? How would you handle it?

Answer: your index formula is only defined for $i < j$. A call with $i > j$ gives a wrong result silently. You must either (a) swap i and j in the macro when $i > j$, or (b) enforce at the call site that queries always use the canonical order. Both add complexity.

Decision: Use full $n \times n$ matrix for Phase 1

Reason: simplicity, debuggability, and uniform access with no conditional in the index. The memory waste is real ($\approx 50\%$) but small in absolute terms at Phase 1 sizes. Pack only if profiling shows memory is a bottleneck, or if you implement an explicit extension.

6 Design Decision 4: Parsing Strategy

6.1 The two parsing approaches

Option A: `fscanf` directly

```c
int id;
double x, y;
while (fscanf(fp, "%d %lf %lf", &id, &x, &y) == 3) {
    // store x, y
}
```

Option B: `fgets` + `sscanf`

```c
char line[256];
while (fgets(line, sizeof(line), fp)) {
    int id;
    double x, y;
    if (sscanf(line, "%d %lf %lf", &id, &x, &y) == 3) {
        // store x, y
    } else {
        // handle malformed line
    }
}
```

6.2 Comparison

Property	`fscanf`	`fgets` + ss
What if a line is 10,000 chars?	Reads until whitespace, no truncation risk	Truncates to buffer
What if a field is "ABC"?	Returns 0 matches; position hard to recover	Line is already buffere
Error recovery position in file	Ambiguous	Clean: you are at
Can you re-examine a bad line?	Only with ungetc tricks	Yes: you have it i
Malformed file: can you free before exit?	Harder (parse state unclear)	Easier (line-level g

Decision: Use `fgets` + `sscanf`

The line-buffer approach keeps the parse state clear at all times. If a line fails to parse, you know exactly where you are in the file, you can log or reject cleanly, and you can call `tsp_instance_free` before returning an error. This is the correct choice for any parser that must handle malformed input gracefully in C.

6.3 One-pass vs. two-pass allocation

When parsing coordinates, you need to allocate arrays. You do not know n until you have scanned the file. Two strategies:

Two-pass: scan the file once counting cities, seek back to the start, allocate exactly, parse again.
One-pass: grow arrays dynamically with `realloc` as you read.

Two-pass or one-pass?

a. What does `realloc` do if the new size is larger? What does it do to the old pointer?

b. In a two-pass parser, what happens if the file is on a pipe or socket rather than disk?

c. Which approach has more predictable memory access patterns during parsing?

d. TSPLIB files often have a `DIMENSION` header field. If you parse it first, can you do single-pass allocation without `realloc`?

Recommended for this assignment: parse the `DIMENSION` header, allocate once with known size, then parse `NODE_COORD_SECTION` in one pass. This avoids both `realloc` complexity and a second file scan.

7 The Complete API Specification

This section gives you complete function contracts. A *contract* specifies: what you must provide (preconditions), what the function guarantees when it returns (postconditions), and what it guarantees on both success and failure paths.

7.1 The data structure (`include/instance.h`)

```c
#ifndef TSP_INSTANCE_H
#define TSP_INSTANCE_H

/* --TSPInstance ------------------------------------------------------------
 *
 * Holds a single TSP instance loaded from a TSPLIB coordinate file.
 *
 * Fields:
 * n Number of cities. Always >= 1 when the struct is valid.
 * Zero indicates an uninitialized or freed struct.
 *
 * coords_x Pointer to an array of n x-coordinates, allocated on the heap.
 * coords_x[i] is the x-coordinate of city i (0-indexed).
 * NULL if not yet allocated.
 *
 * coords_y Pointer to an array of n y-coordinates, allocated on the heap.
 * coords_y[i] is the y-coordinate of city i (0-indexed).
 * NULL if not yet allocated.
 *
 * dist Pointer to a flat row-major n*n distance matrix, heap-allocated.
 * dist[i*n + j] is the Euclidean distance from city i to city j.
 * NULL if not yet built.
 *
 * Invariants (when valid):
 * dist[i*n + i] == 0.0 for all i
 * dist[i*n + j] == dist[j*n+i] for all i, j (symmetry)
 * dist[i*n + j] >= 0.0 for all i, j
 *
 * Ownership:
 * The struct owns all three pointers. tsp_instance_free() must be called
 * exactly once on any struct touched by tsp_instance_load(), whether or
 * not the load succeeded.
 * ---------------------------------------------------------------------------*/
typedef struct {
    int n;
    double *coords_x;
    double *coords_y;
    double *dist;
} TSPInstance;

/* Access macro. i and j must be in [0, inst->n). */
#define DIST(inst, i, j) ((inst)->dist[(i) * (inst)->n + (j)])

/* --Function declarations ---------------------------------------------------*/

```

```c
int tsp_instance_load(const char *path, TSPInstance *out);
void tsp_instance_free(TSPInstance *inst);
int tsp_build_distance_matrix(TSPInstance *inst);

#endif /* TSP_INSTANCE_H */
```

API: `tsp_instance_load`

Signature:

```c
int tsp_instance_load(const char *path, TSPInstance *out);
```

Arguments:

- `path` — A null-terminated C string giving the file system path to a TSPLIB-format `.tsp` file. Must not be `NULL`.
- `out` — Pointer to a `TSPInstance` that the caller provides. The function writes the loaded instance into this struct. Must not be `NULL`. The struct's contents before the call are ignored; the function overwrites them.

Returns:

- `0` on success. `out` is fully populated: `n` ≥ 1, `coords_x` and `coords_y` are heap-allocated arrays of length `n`, `dist` is `NULL` (matrix is not built by this function).
- `-1` on any failure. Failure conditions include: file not found, file is empty, `NODE_COORD_SECTION` is missing, any coordinate line is malformed, any allocation fails. On failure, `out` is left in a state that is safe to pass to `tsp_instance_free()`: all pointers it owns are either valid or `NULL`.

What it does (step by step):

1. Initialise `*out` to the zero struct (`n=0`, all pointers `NULL`). This ensures `free` is safe even if we return early.
2. Open the file at `path`. Return `-1` if it cannot be opened.
3. Scan lines until `DIMENSION: N` (or `DIMENSION : N`) is found. Parse N as the city count. Return `-1` if not found before `NODE_COORD_SECTION` or EOF.
4. Allocate `coords_x[N]` and `coords_y[N]`. Return `-1` if either allocation fails.
5. Scan lines until `NODE_COORD_SECTION` is found. Return `-1` if not found.
6. For each of the next N lines, parse with `sscanf(line, "%d %lf %lf", &id, &x, &y)`. If any line does not produce exactly 3 fields, return `-1`.
7. Store `x` in `coords_x[id-1]` and `y` in `coords_y[id-1]`. (TSPLIB IDs are 1-indexed; convert to 0-indexed.)
8. Close the file. Set `out->n = N`. Return `0`.

What it does NOT do:

- It does *not* build the distance matrix. Call `tsp_build_distance_matrix` separately.
- It does *not* validate that city IDs form a contiguous sequence starting at 1.

API: `tsp_instance_free`

Signature:

```
void tsp_instance_free(TSPInstance *inst);
```

Arguments:

- `inst` — Pointer to a `TSPInstance`. May be `NULL` (the function is a no-op in that case). The struct it points to may be partially initialised (some pointers non-`NULL`, others `NULL`).

Returns: Nothing.
What it does:

1. If `inst` is `NULL`, return immediately.
2. Call `free(inst->coords_x)` if it is not `NULL`. Set to `NULL`.
3. Call `free(inst->coords_y)` if it is not `NULL`. Set to `NULL`.
4. Call `free(inst->dist)` if it is not `NULL`. Set to `NULL`.
5. Set `inst->n = 0`.

Why nullify after free? Calling `free()` on a pointer does not set the pointer to `NULL`. If anything later accidentally calls `free()` again on the same pointer, it is undefined behaviour (double-free). Setting to `NULL` after freeing makes the struct safe to call `free` on a second time (`free(NULL)` is always a no-op in C).
Why safe on partial init? During `tsp_instance_load`, several allocations happen. If the third allocation fails and you return `-1`, the struct has some non-`NULL` pointers and some `NULL` ones. If `free` only frees non-`NULL` pointers, the caller can always call `tsp_instance_free` unconditionally without leaking.

API: `tsp_build_distance_matrix`

Signature:

```
int tsp_build_distance_matrix(TSPInstance *inst);
```

Arguments:

- `inst` — Pointer to a `TSPInstance` that has already been successfully loaded: `inst->n` ≥ 1, `coords_x` and `coords_y` are valid, `dist` is `NULL` (or the function will overwrite it).

Returns:

- `0` on success. `inst->dist` points to a heap-allocated array of `n*n` doubles satisfying the invariants (diagonal zero, symmetric, non-negative).
- `-1` if `malloc` fails. `inst->dist` is left `NULL`.

What it does:

1. Allocate `n*n` doubles. Return `-1` on failure.
2. For all i: set `DIST(inst,i,i) = 0.0`.
3. For all $i < j$: compute $d = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ using `sqrt()` from `<math.h>`. Set both `DIST(inst,i,j) = d` and `DIST(inst,j,i) = d`.
4. Return `0`.

Why only the upper triangle? Computing only $j > i$ means each pair is computed once. Mirroring enforces $D_{ij} = D_{ji}$ *by construction* from the same value, not from two independent computations that might differ by a floating-point epsilon.

The `sqrt` vs `sqrtf` issue: `sqrt()` from `<math.h>` takes and returns `double`. `sqrtf()` takes and returns `float`. Since both coordinates and distances are `double` in this design, use `sqrt()`. If you used `float` and called `sqrt()` instead of `sqrtf()`, the argument would be implicitly promoted to `double` for the computation and then truncated back to `float` on assignment—not undefined behaviour, but needless and slightly imprecise. Always match the function suffix to your type.

8 The TSPLIB File Format (Narrow Subset)

You will support one strict subset of TSPLIB. Not all variants. This is intentional: a parser that handles one format correctly is better than a parser that handles ten formats inconsistently.

8.1 Format you will support

```
NAME: square_4
TYPE: TSP
COMMENT: 4-city unit square
DIMENSION: 4
EDGE_WEIGHT_TYPE: EUC_2D
NODE_COORD_SECTION
1 0.0 0.0
2 1.0 0.0
3 1.0 1.0
4 0.0 1.0
EOF
```

Rules your parser enforces:

- Header fields may appear in any order before `NODE_COORD_SECTION`.
- `DIMENSION` must appear and must be a positive integer.
- `NODE_COORD_SECTION` must appear.
- After `NODE_COORD_SECTION`, each line must parse as `int double double` (id, x, y).
- The parser reads exactly `DIMENSION` coordinate lines.
- `EOF` at end is optional; the parser stops when it has read all n cities.

What you do NOT need to handle (yet):

- `EDGE_WEIGHT_SECTION` (explicit weight matrices)
- `DISPLAY_DATA_SECTION`
- `EDGE_WEIGHT_TYPE` values other than `EUC_2D`
- 3D coordinates

8.2 Test fixture: `square_4.tsp`

Create this file exactly. You will use it to verify every geometric invariant.

```
NAME: square_4
TYPE: TSP
DIMENSION: 4
EDGE_WEIGHT_TYPE: EUC_2D
NODE_COORD_SECTION
1 0.0 0.0
2 1.0 0.0
3 1.0 1.0
4 0.0 1.0
EOF
```

Cities 0–3 (0-indexed after loading) form a unit square. The expected distance matrix is:

$$D = \begin{pmatrix} 0 & 1 & \sqrt{2} & 1 \\ 1 & 0 & 1 & \sqrt{2} \\ \sqrt{2} & 1 & 0 & 1 \\ 1 & \sqrt{2} & 1 & 0 \end{pmatrix}$$

9 C Implementation Guide

This section gives you enough C scaffolding that you can implement each function without guessing syntax, while still writing the logic yourself.

9.1 Step 1: Struct, macro, and static initialiser

In `include/instance.h`, place the struct and macro as shown in Section 6.

In `src/instance.c`, write a static helper to initialise a struct to the safe zero state:

```c
#include "instance.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

/* Zero-initialise: all pointers NULL, n = 0.
 * After this call, tsp_instance_free(inst) is always safe. */
static void tsp_instance_init(TSPInstance *inst)
{
    inst->n = 0;
    inst->coords_x = NULL;
    inst->coords_y = NULL;
    inst->dist = NULL;
}
```

Write this first. Every other function depends on this invariant.

9.2 Step 2: Implement `tsp_instance_free`

Before writing the parser, write cleanup. The reason is: every early-return error path in the parser will call `tsp_instance_free`. If cleanup is buggy, your error paths are buggy.

Pattern to follow:

```c
void tsp_instance_free(TSPInstance *inst)
{
    if (inst == NULL) return;
    free(inst->coords_x); inst->coords_x = NULL;
    free(inst->coords_y); inst->coords_y = NULL;
    free(inst->dist); inst->dist = NULL;
    inst->n = 0;
}
```

Why is `free(NULL)` safe?

The C standard (C99 §7.20.3.2) guarantees that `free(NULL)` has no effect. This means you never need to check for `NULL` before calling `free`. But you *do* need to set pointers to `NULL` after freeing them, to prevent accidental double-free.

9.3 Step 3: Skeleton of `tsp_instance_load`

```c
int tsp_instance_load(const char *path, TSPInstance *out)
{
    tsp_instance_init(out); /* safe even if we return -1 early */

    FILE *fp = fopen(path, "r");
    if (fp == NULL) return -1;

    char line[512];
    int dimension_found = 0;
    int n = 0;

    /* --Pass 1: scan headers for DIMENSION ------------------------*/
    /* YOUR CODE: loop with fgets, look for "DIMENSION" keyword,
       parse the integer, set dimension_found = 1 when found.
       Stop when you hit NODE_COORD_SECTION or EOF. */

    if (!dimension_found) { fclose(fp); return -1; }

    /* --Allocate coordinate arrays --------------------------------*/
    out->coords_x = malloc(n * sizeof(double));
    out->coords_y = malloc(n * sizeof(double));
    if (!out->coords_x || !out->coords_y) {
        fclose(fp);
        tsp_instance_free(out); /* free whatever was allocated */
        return -1;
    }

    /* --Find NODE_COORD_SECTION -----------------------------------*/
    /* YOUR CODE: continue scanning lines until NODE_COORD_SECTION.
       If not found before EOF, call tsp_instance_free and return -1. */

    /* --Parse coordinate lines ------------------------------------*/
    for (int k = 0; k < n; k++) {
        if (fgets(line, sizeof(line), fp) == NULL) {
            fclose(fp); tsp_instance_free(out); return -1;
        }
        int id;
        double x, y;
        /* YOUR CODE: sscanf the line. Check return value == 3.
           Store: out->coords_x[id-1] = x; out->coords_y[id-1] = y;
           If sscanf fails, call tsp_instance_free and return -1. */
    }

    fclose(fp);
    out->n = n;
    return 0;
}
```

9.4 Step 4: Implement `tsp_build_distance_matrix`

```c
int tsp_build_distance_matrix(TSPInstance *inst)
{
```

```c
    int n = inst->n;
    inst->dist = malloc((size_t)n * n * sizeof(double));
    if (inst->dist == NULL) return -1;

    for (int i = 0; i < n; i++) {
        DIST(inst, i, i) = 0.0;
        for (int j = i + 1; j < n; j++) {
            double dx = inst->coords_x[i] - inst->coords_x[j];
            double dy = inst->coords_y[i] - inst->coords_y[j];
            double d = sqrt(dx*dx + dy*dy);
            DIST(inst, i, j) = d;
            DIST(inst, j, i) = d;
        }
    }
    return 0;
}
```

Note: `(size_t)n * n` is important. If `n` is large enough that `n*n` overflows a 32-bit `int`, the multiplication is done as `int` and wraps before the cast. Casting one operand to `size_t` first forces the multiplication to happen in 64-bit.

9.5 Step 5: The Makefile

```makefile
CC = gcc
CFLAGS = -Wall -Wextra -Werror -std=c11 -Iinclude
LDFLAGS = -lm

all: test_instance

test_instance: src/instance.c tests/test_instance.c
	$(CC) $(CFLAGS) $^ $(LDFLAGS) -o $@

clean:
	rm -f test_instance

.PHONY: all clean
```

10 Testing: Writing Plain-C Unit Tests

You are not using a test framework. You are writing a test runner in plain C. Here is the minimal pattern:

```c
/* tests/test_instance.c */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "instance.h"

/* --Test helpers ------------------------------------------------------------*/
static int tests_run = 0;
static int tests_passed = 0;

#define CHECK(condition, msg) \
    do { \
        tests_run++; \
        if (condition) { \
            tests_passed++; \
        } else { \
            fprintf(stderr, "FAIL [%s:%d] %s\n", \
                    __FILE__, __LINE__, msg); \
        } \
    } while (0)

#define CHECK_NEAR(a, b, tol, msg) CHECK(fabs((a)-(b)) < (tol), msg)

/* --Tests -------------------------------------------------------------------*/

static void test_load_square(void)
{
    TSPInstance inst;
    int rc = tsp_instance_load("tests/fixtures/square_4.tsp", &inst);

    CHECK(rc == 0, "P-01: load returns 0 on success");
    CHECK(inst.n == 4, "P-02: city count is 4");

    if (rc != 0) { tsp_instance_free(&inst); return; }

    /* P-03: spot-check one coordinate */
    CHECK_NEAR(inst.coords_x[0], 0.0, 1e-9, "P-03: city 0 x == 0.0");
    CHECK_NEAR(inst.coords_y[0], 0.0, 1e-9, "P-03: city 0 y == 0.0");

    int mrc = tsp_build_distance_matrix(&inst);
    CHECK(mrc == 0, "matrix build returns 0");

    /* D-01: diagonal is zero */
    for (int i = 0; i < inst.n; i++)
        CHECK(DIST(&inst, i, i) == 0.0, "D-01: diagonal is zero");

    /* D-02: symmetry */
    for (int i = 0; i < inst.n; i++)
        for (int j = 0; j < inst.n; j++)
```

```c
            CHECK_NEAR(DIST(&inst, i, j), DIST(&inst, j, i), 1e-12, "D-02: symmetric");

    /* D-03: adjacent sides == 1 */
    CHECK_NEAR(DIST(&inst, 0, 1), 1.0, 1e-9, "D-03: dist(0,1)==1");
    CHECK_NEAR(DIST(&inst, 1, 2), 1.0, 1e-9, "D-03: dist(1,2)==1");

    /* D-04: diagonal == sqrt(2) */
    CHECK_NEAR(DIST(&inst, 0, 2), sqrt(2.0), 1e-9, "D-04: dist(0,2)==sqrt(2)");
    CHECK_NEAR(DIST(&inst, 1, 3), sqrt(2.0), 1e-9, "D-04: dist(1,3)==sqrt(2)");

    tsp_instance_free(&inst);
    /* M-01: after free, pointers are NULL */
    CHECK(inst.coords_x == NULL, "M-01: coords_x NULL after free");
    CHECK(inst.dist == NULL, "M-01: dist NULL after free");
}

static void test_malformed_bad_coord(void)
{
    TSPInstance inst;
    int rc = tsp_instance_load("tests/fixtures/malformed_bad_coord.tsp", &inst);
    CHECK(rc == -1, "F-01: malformed coord returns -1");
    /* Must be safe to free even after failure */
    tsp_instance_free(&inst);
    CHECK(inst.n == 0, "F-01: n is 0 after failed load + free");
}

static void test_missing_section(void)
{
    TSPInstance inst;
    int rc = tsp_instance_load("tests/fixtures/malformed_missing_section.tsp", &inst);
    CHECK(rc == -1, "F-02: missing section returns -1");
    tsp_instance_free(&inst);
}

/* --Main ------------------------------------------------------------------*/
int main(void)
{
    test_load_square();
    test_malformed_bad_coord();
    test_missing_section();

    printf("\nResults: %d/%d tests passed\n", tests_passed, tests_run);
    return (tests_passed == tests_run) ? 0 : 1;
}
```

10.1 Malformed fixture files

malformed_bad_coord.tsp:

```
NAME: bad_coord
DIMENSION: 2
NODE_COORD_SECTION
1 0.0 0.0
2 NOTANUMBER 1.0
```

```
EOF
```

malformed_missing_section.tsp:

```
NAME: missing_section
DIMENSION: 2
EOF
```

10.2 Running under Valgrind

```
make
valgrind --leak-check=full --error-exitcode=1 ./test_instance
```

Valgrind should report:

```
==PID== HEAP SUMMARY:
==PID==     in use at exit: 0 bytes in 0 blocks
==PID==  LEAK SUMMARY:
==PID==     definitely lost: 0 bytes in 0 blocks
```

If you see leaks, trace them by adding `-track-origins=yes`.

11 Test Matrix: What You Are Verifying

ID	Category	What it checks
P-01	Parsing	`tsp_instance_load` returns 0 on a valid file
P-02	Parsing	Loaded city count matches `DIMENSION`
P-03	Parsing	At least one coordinate value matches expected
D-01	Matrix	`DIST(inst, i, i) == 0.0` for all i
D-02	Matrix	`DIST(inst, i, j) == DIST(inst, j, i)` for all i, j
D-03	Matrix	Adjacent square sides equal 1.0 within tolerance
D-04	Matrix	Square diagonals equal $\sqrt{2}$ within tolerance
F-01	Failure	Malformed coordinate line returns -1
F-02	Failure	Missing `NODE_COORD_SECTION` returns -1
M-01	Memory	All pointers are `NULL` after `tsp_instance_free`
M-02	Memory	Valgrind reports zero leaked bytes on success path
M-03	Memory	Valgrind reports zero leaked bytes on failure path

12 Questions You Must Answer in Writing

Answer these in a `DESIGN_NOTES.md` file or as block comments at the top of `src/instance.c`. These are not optional. The point is to force you to connect the code you wrote to the reasoning behind it.

Q1. Why does `tsp_instance_free` set pointers to `NULL` after calling `free`? What specific undefined behaviour does this prevent?

Q2. Why is `tsp_instance_init` called at the very beginning of `tsp_instance_load`, before the file is opened?

Q3. Why does `tsp_build_distance_matrix` iterate only for $j > i$ and then assign both `DIST(inst,i,j)` and `DIST(inst,j,i)` from the same value, rather than computing all n^2 entries independently?

Q4. If you had used `double **dist` instead of `double *dist`, how many `free()` calls would `tsp_instance_free` need to make? Write out the loop.

Q5. A colleague proposes: "We should use `float` for the distance matrix so it fits in GPU constant memory." Walk through the arithmetic for $n = 128$ and explain why this argument is not as clean as it sounds.

Q6. The `DIST` macro has parentheses around every argument. Write an example where omitting the parentheses around a single argument would produce a silently wrong result.

Q7. At what point in the code does the struct transition from "partially initialised" to "fully valid"? Identify the exact line.

Q8. Why does `tsp_build_distance_matrix` cast to `(size_t)` before multiplying `n * n`? At what value of n would the multiplication overflow a 32-bit signed integer?

13 Implementation Order Summary

Follow this order. Do not skip steps.

Step 1. Write `include/instance.h`: struct, macro, declarations.

Step 2. Write `src/instance.c`: `tsp_instance_init` (static), `tsp_instance_free`.

Step 3. Compile with an empty `main`. Fix all warnings before proceeding.

Step 4. Write `tsp_instance_load`: open, scan headers, parse `DIMENSION`, allocate, find `NODE_COORD_SECTION`, parse coordinates.

Step 5. Create the test fixtures.

Step 6. Write `tests/test_instance.c`: `test_load_square` only.

Step 7. Compile and run. Fix failures.

Step 8. Write `tsp_build_distance_matrix`.

Step 9. Add matrix tests (D-01 through D-04). Fix failures.

Step 10. Add failure-mode tests (F-01, F-02). Fix failures.

Step 11. Run under Valgrind. Fix all leaks before proceeding.

Step 12. Write your design notes answering Q1–Q8.

14 Reflection: After Everything Passes

Once Valgrind is clean and all tests pass, answer these reflection questions. These bridge Phase 1 to the GPU work ahead.

1. If the instance size grew to 5000 cities, the full matrix would be $5000^2 \times 8 = 200\,\text{MB}$. What options would you have? Name at least two distinct strategies.

2. GPU global memory is large but has high latency. GPU shared memory is fast but limited ($\approx$48 KB per thread block on modern hardware). When the GA kernel accesses the distance matrix, what access pattern determines whether shared memory can help?

3. Your current parser is strict: it only handles one file format subset. If you later need to support TSPLIB files with explicit edge weight matrices (`EDGE_WEIGHT_SECTION`), what part of `tsp_instance_load` would change? What part would stay the same?

4. `tsp_build_distance_matrix` and `tsp_instance_load` are currently separate functions. Describe one scenario where that separation would make your Phase 2 implementation significantly easier.

5. Write one sentence describing why `free(NULL)` being a no-op is a deliberate design decision in the C standard, not an accident.

15 Optional Extensions

Do these only after all tests pass and the design notes are written.

15.0 Extension A: Packed symmetric storage

Store only entries with $i < j$. The index formula for $(i < j)$ in a packed array is:

$$\text{idx}(i, j) = i \cdot n - \frac{i(i+1)}{2} + j - i - 1$$

- Implement a `DIST_PACKED` macro that handles both $i < j$ and $i > j$.
- Rerun all tests. Do any break?
- Measure: how much less memory is used for $n = 512$?

15.0 Extension B: `float` distance matrix

Switch `dist` from `double` to `float`. Use `sqrtf` instead of `sqrt`.

- Adjust test tolerances where needed.
- For $n = 128$, compare matrix footprint against constant-memory budget.
- Answer: does this change improve anything measurable on CPU?

15.0 Extension C: Command-line loader

Write a small `src/main.c` that accepts a `.tsp` file as a command-line argument, loads it, builds the matrix, and prints the matrix dimensions and the distance from city 0 to city 1.

```
./tsp_info tests/fixtures/square_4.tsp
# Loaded 4 cities.
# dist(0,1) = 1.000000
```